

Vector3, we would be taken to the documentation page for vector3. The same is true for ForceMode, MonoBehaviour and Rigidbody. Below the description are snippets that show us how this function could be used. Note the second signature for AddForce below the first. The descriptions are the same. But in this case we can use AddForce with either a vector3 or 3 float values for X, Y and Z. The next concept that we need to cover is how to get a hold of, or how to reference different components on our game object. We are writing a script called PlayerController. Which is attached as a script component to our Player game object. From this script we need to AddForce using the rigidbody component. We want to access that component from this script. There are several ways that we can do this. But in our case we will cover only one of the main ways of accessing another component on the same game object. We will create a variable to hold this reference in our script and we will set this reference in the Start function. We see this here in the code snippet. public Rigidbody rb creates a public variable with the type of rigidbody called rb to hold the reference to the rigidbody we want to access. In Start the reference is set by using the code GetComponent This will find and return a reference to the attached rigidbody, if there is one. All of the code in the start function is called on the first frame that the script is active. This is often the very first frame of the game. Finally in FixedUpdate the attached rigidbody component is accessed through the variable named rb with rb.AddForce. So in our script we need to write private Rigidbody rb to create the variable to hold the

```
using UnityEngine;
using System.Collections;

public class ExampleClass : MonoBehaviour {
    public float thrust;
    public Rigidbody rb;
    void Start() {
        rb = GetComponent<Rigidbody>();
    }
    void FixedUpdate() {
        rb.AddForce(transform.forward * thrust);
    }
}
```

Referencing the rigid body